llmctl docs/decide-gateway

Decide gateway reference (`llmctl decide serve`)

Revision: 4 **Last modified:** 2026-10-08T00:00:00Z

`llmctl decide serve` runs the **Go decision gateway** (`cmd/llmctl-decide` , `internal/server` , `internal/gateway` , `internal/contract`): one HTTPS server that exposes the local decision models over the Jev / TypeSafe-SDK wire shape (`POST /v1/systemone`). Default port **8095**; TLS 1.2+ with a local CA (`llmctl decide cert`), a mandatory access key (`llmctl decide key`), and engines that stay on loopback behind it.

> **History.** The first candidate of this feature shipped a stdlib Python gateway (`lib/decide_gateway.py` , plain HTTP, optional key, `--backend-engine` flag). It was **never committed and never in any tag**; it was superseded by the Go gateway and retired after assertion-by-assertion parity was shown (`specs/009-jev-decision-models/evidence/python-gateway-parity.json`). The retired files are archived, with a git-history note, under `specs/009-jev-decision-models/evidence/python-gateway-retired/` . Nothing in this document describes that Python gateway any more.

Everything below is exercised by `tests/test_gateway_endpoints.sh` (the built binary over real HTTPS against a Go fake engine: every row of `specs/009-jev-decision-models/contracts/endpoint-inventory.tsv`), the Go tests of `internal/server` , `internal/gateway` , `internal/contract` , and `cmd/llmctl-decide` , and `tests/test_decide_cli.sh` (the shell front end against a real TLS gateway).

Request flow (diagram)

Source: `diagrams/decide-gateway-flow.mmd` (rendered svg). The transport is HTTPS only and every `/v1/*` request needs the bearer key (see Auth below).

```
sequenceDiagram
  autonumber
  participant C as Client (TypeSafe SDK, coding agent, claude_toolkit JEV MCP)
  participant G as llmctl-decide gateway (HTTPS, default port 8095)
  participant A as Auth (bearer key, constant-time)
  participant R as Registry and resolver (validated profiles only)
  participant E as Engine on loopback (llama-server or onnx)
  C->>G: POST /v1/systemone over HTTPS (local CA, TLS 1.2+), Authorization Bearer key
  G->>A: check the bearer key
  alt key missing or wrong
    A-->>C: 401 (repeated failures: 429 rate_limited)
  else key valid
    A->>R: pick a profile (explicit, or auto ranking)
    R->>R: llmctl require-validated gate (strict by default)
    alt profile not validated
      R-->>C: refused, no engine call
    else validated
      R->>E: typed decision request (internal key file, loopback only)
      E-->>G: logprob readout
      G-->>C: 200 typed answer (choice, noul or score)
    end
  end
```

Governance (Constitution §11.4.279, §11.4.277, claude_toolkit)

The decide gateway is llmctl's side of the decisions-model layer (Constitution §11.4.279; the anchor text lives in the constitution, this page describes only what llmctl implements): typed decisions over HTTPS with a mandatory bearer key, served only from profiles that pass the validation gate (§11.4.277, validation). llmctl is a constitution hub engine (dependency layout, §11.4.28(C) / §11.4.31). The **consumer** side is claude_toolkit's JEV integration (`jev-setup.sh` , `jev-gate.sh` , `jev_mcp.py`): it registers decision endpoints as

`classification: "decision"` (never aliases) and sends `Authorization: Bearer <key>`; a consumer must use the `https://` base URL and trust the local CA. That the toolkit's scripts handle the HTTPS scheme and CA correctly end to end is not verified in this repository.

Lifecycle

```
llmctl decide serve [--port N] [--bind H]
        # detached; prints URL, key LOCATION, CA fingerprint
llmctl decide serve --foreground                # exec in place (what
        the systemd/launchd unit runs)
llmctl decide serve --status                    # rc 0 only for a
        verified, running gateway
llmctl decide serve --stop                      # signals only a
        process verified to be this gateway
llmctl decide serve --enable [--now]            # persistent boot-time
        user service (systemd user unit / launchd agent)
llmctl decide serve --disable                   # stop, disable and
        remove that service
```

- The Go binary is built once by `llmctl build decide` (`LLMCTL_DECIDE_BIN` overrides the path); the shell front end builds it on first use when Go is installed and says exactly what to run when it is not.
- State: pidfile `$LLMCTL_STATE_DIR/decide/gateway.pid` (mode 0600), request log `$LLMCTL_LOG_DIR/decide-requests.jsonl` (mode 0600; ts, request id, status, ms, bytes, auth result, profile, TLS version, **keyed state hash** - never state text or key material). An opt-in **decision log** is a separate file, see "Calibration and the decision log".
- `--stop` never signals an unverified process (Helix 11.4.263): a pidfile naming an unrelated process, or pid 1, is refused (rc 1); `--stop` with no pidfile is a clean no-op (rc 0). `--status` does not report an unverified pid as running.
- It starts with **no key or no valid certificate refused** (rc 4 / 5): the key is generated on first start and persisted to the installation `.env` (mode 0600); the certificate chain is created under `$LLMCTL_HOME/cert`.
- The gateway does not start engines: engines are scheduled by `llmctl enable` / `auto` (or found in the registry, see "Finding engines"); a profile without a ready engine answers `503 not_ready` instead of starting one.

- `--enable` is idempotent on Linux (on macOS it boots the agent out first; UNCONFIRMED on real launchd), applies a changed unit file with `try-restart`, refuses while an installed engine unit is stale (`llmctl install`) and starts the service at once (`--now` spells that out); `--disable` removes it. Registered engines are probed at the path they serve: llama `/health`, onnx `/readyz` (truthful readiness; `/healthz` is liveness only), colibri `/v1/models` (a wrong path makes the registry mark the entry unhealthy and remove it after the reconciler grace, 30 s by default).
- On boot it runs as a user service (`llmctl-decide-gateway.service` / launchd agent) whose wrapper allocates the port and publishes the gateway in the registry (`docs/registry-discovery.md`).
- `LLMCTL_DRY_RUN=1 llmctl decide serve ...` prints the delegation and starts nothing.
- Persistent operation (linger, `gateway.conf`, drop-ins, stop behaviour): persistent-services. Serving other machines: lan-exposure. Certificate/key procedures: runbooks.

Engines are chosen by the catalog, not by a flag

Each decision profile in `models/catalog.json` names its `decision.protocol`; the router picks the driver from it:

Pro-tocol	Engine	How the gateway answers
`letter-logit`	`llama-server` (GGUF decoder)	renders the shared lettered-option prompt, asks for ONE token with first-token logprobs, reads the option-letter probabilities (`internal/readout`), renormalises
`nli-onnx`	encoder runtime `lib/onnx_server.py` (`onnx` engine)	sends one premise/hypothesis pair per option to the runtime's `POST /v1/score` and normalises the entailment column

Pro-tocol	Engine	How the gateway answers
`syste-mone-native`	an engine with a native `/v1/systemone` (disabled until the engine-advance gate passes)	proxies and **re-validates/re-shapes** the engine's answer through the con-tract

There is no `--backend-engine` flag; typed-question logic lives only in Go (one implementation per rule).

Endpoints

`POST /v1/systemone`

Request (TypeSafe SDK shape) - `{model, state, questions}` ; `questions` maps a name to `{type, instructions, criteria}` :

```
{
  "model": "jev-latest",
  "state": "Routing.",
  "questions": {
    "team": {"type": "choice", "instructions": "Which team handles
        invoices?",
            "criteria": {"billing": "handles invoices", "legal":
        "contracts"}}
  }
}
```

Response 200 (the shape asserted by `internal/contract` `TestResponseEnvelope`):

```
{"model":"decide-tiny",
 "answers":{"team":{"type":"choice","choice":"billing","probabilities":
        {"billing":0.96,"legal":0.04},"confidence":0.93}},
 "usage":{"input_tokens":61,"output_tokens":1}}
```

(Probabilities are illustrative; tests compute the expected values from the fixture logprobs.)

Semantics, all enforced in code:

- **Question names are never rendered into the prompt**; they only key `answers`.
- `model` in the response is always the **served profile id** (never the request's string); hosted aliases (`jev-latest`, `jev-1.13.0`, `jev-preview`) and `llmctl-<profile>` resolve to a served profile. An unknown or non-string model is `422 unknown_model`, never echoed.
- `noul`: `{"type","noul"}` with the yes-probability, **no confidence**; `choice`: `choice`, `probabilities` (option order), `confidence`; `score`: weighted expectation, `legend`, `probabilities`, `confidence`. Probabilities are finite, sum to 1 and are rounded to 9 decimals.
- `state` may be a string, object or array (serialised deterministically); `questions` is a non-empty object (at most `LLMCTL_DECIDE_MAX_QUESTIONS`, default 32); options are capped per profile (decoder profiles 20 by default, hard cap 26 letters; native/encoder profiles up to the hosted 255).
- A model that answers something other than an option letter, or whose letter mass is below the threshold, is `422 readout_failed` (deterministic, not retryable) - never a renormalised guess.
- Option-forging text inside `state` is neutralised before it reaches the prompt.
- `usage.input_tokens` is an **estimate** (`ceil(characters/4)` over the rendered prompts); `usage.output_tokens` is exact for decoder profiles (one token per question).
- Errors are `{"message","error_type"}`: `400 invalid_request`, `401 unauthorized`, `405 method_not_allowed`, `413 payload_too_large`, `422 validation_failed|unknown_model|readout_failed`, `429 rate_limited` (failed-auth bursts only), `502 backend_failed` (generic - engine text never reaches a client), `503 not_ready`, `529 overloaded`.

`GET /v1/models`

One body, two listings: the `object` / `data` list (id, aliases, protocol, status, limits, notes) and the hosted SDKs' `{"models":[{"name","description","release_date"}]}` - one entry per profile id and per alias - because `typesafe-sdk` 0.7.2 and `@typesafe-ai/sdk` 0.6.0 reject any other shape. `description` is the catalog `desc`; `release_date` is the optional `decision.release_date`, else the fixed documented default. Requires the key.

`GET /healthz`, `GET /readyz`, `GET /metrics`

- `/healthz` -> `200 {"status":"ok"}` (liveness, **auth-exempt**, minimal: it probes nothing and leaks no profile name).
- `/readyz` -> `200 {"status":"ready"}` when at least one engine is routable, else `503 not_ready`; auth-exempt.
- `/metrics` -> bounded Prometheus text (no state, no keys); requires the key.

TypeSafe SDK setup

```
export TYPESAFE_BASE_URL=https://127.0.0.1:8095
export TYPESAFE_API_KEY="$LLMCTL_API_KEY"          # llmctl decide key
        show --yes-print shows it deliberately
export SSL_CERT_FILE="$LLMCTL_HOME/cert/ca/ca.crt"  # CA trust
        (Python); NODE_EXTRA_CA_CERTS for Node
```

The gateway speaks the request/response wire shape above; it is **not** a byte-for-byte clone of the hosted Jev API (see "Token-usage honesty" and the FAQ). Trust is the local CA only; there is no switch that disables certificate verification.

Auth

- One access key (`LLMCTL_API_KEY`, resolved from the environment, then the installation `.env`, else generated on first start). Every `/v1/*` request

needs `Authorization: Bearer <key>` (or `x-api-key` when enabled); absent/wrong -> `401`.

- The key is **never** on a command line and never printed except by `llmctl decide key show --yes-print`. Comparison is constant-time. Rotation: `llmctl decide key rotate [--grace N]` (accepted keys are re-read; no restart).
- Repeated failed authentications from one source are labelled `429 rate_limited` (`LLMCTL_DECIDE_AUTH_FAIL_LIMIT` per minute) and every further failure in the window is delayed a little more (25 ms per failure, at most 250 ms); a request carrying the valid key is never throttled **and never delayed**. The throttle does **not** make guessing impossible (a correct guess still succeeds - the valid key must never be locked out), so security rests on key entropy: an operator-supplied key is refused unless it has at least 8 distinct characters, no character dominating, no repeated block and about 128 estimated bits (generated keys are 256 bits and always pass). Details: `docs/tls-and-keys.md`.
- Revoking a key: `llmctl decide key rotate` (without `--grace`) and restart, or remove it from the key file - when the key source becomes unreadable, unsafe or empty the gateway keeps the previous keys for 5 seconds, says so once on its stderr, counts `llmctl_decide_key_source_errors_total` and then **refuses every request** until the source is usable again.
- The key the gateway presents to **engines** is separate and travels in a 0600 file (`LLMCTL_DECIDE_INTERNAL_KEY_FILE` or `$LLMCTL_STATE_DIR/keys/<kind>-<profile>.key`), never in the environment or argv.

State budget and truncation

`LLMCTL_DECIDE_MAX_STATE_CHARS` (default 8192) is the state budget for decoder profiles. **An over-budget state is rejected with `422 validation_failed` by default.** With the opt-in `LLMCTL_DECIDE_TRUNCATE=1` the state is shortened (head and tail kept) and the response carries `x-llmctl-decide-truncated: true`; the header is also set when an encoder runtime

reports that it truncated a premise (opt-in; otherwise 422). Question and option text are never shortened.

Slow prefill and the deadline

`422 validation_failed` is only for **over-budget input** (the token estimate exceeds what the serving instance's context allows). An input that *fits* the context but whose prefill takes longer than the end-to-end deadline (`LLMCTL_DECIDE_TIMEOUT` , default 8 s - e.g. several thousand dense hex characters on a large-context profile) is answered `502 backend_failed` with the additive headers `x-llmctl-decide-reason: deadline_exceeded` and `x-llmctl-decide-deadline-ms: <deadline>` ; any other backend-call 502 carries `x-llmctl-decide-reason: engine_error` (the two gateway-internal 502 paths - an unclassifiable request-parse error and a response-marshalling failure - set no reason header). The error body is unchanged. The deadline is a limit, not a guarantee that an input within the context budget is accepted in time: send a smaller state, use a faster profile, or raise the deadline.

Concurrency and limits

`LLMCTL_DECIDE_CONCURRENCY` (4) simultaneous engine calls with a bounded `LLMCTL_DECIDE_QUEUE` (64) -> `529 overloaded`

- `Retry-After` ; per-instance slots keep deterministic mode single-slot per engine. Connection admission, handshake/read deadlines, the end-to-end `LLMCTL_DECIDE_TIMEOUT` and a graceful drain on stop are all in `specs/009-jev-decision-models/contracts/env-vars.md` and `docs/user-manual.md` .

The end-to-end budget and slow (CPU) engines

`LLMCTL_DECIDE_TIMEOUT` (seconds, Go default `8` , kept below the hosted SDK's 10 s timeout; raised to `120` by the launcher when a CPU-placed decision engine is served and nothing is set explicitly, see "CPU-adaptive default" below) is the ONE deadline of a request: queue wait plus the engine call(s). There is no separate per-attempt deadline and the gateway does not

retry an engine call, with one exception: after an engine answers `401` it re-reads the engine key file once and retries once with the new key (`internal/gateway/driver.go` ; never with the same key, never a loop). When the budget expires the engine connection is closed (the engine logs `cancel task`) and the client gets `502 backend_failed` (the documented status; unchanged) with the additive headers `x-llmctl-decide-reason: deadline_exceeded` and `x-llmctl-decide-deadline-ms: <budget>` . Every other backend-call 502 carries `x-llmctl-decide-reason: engine_error` (the gateway-internal parse/marshal 502 paths set no reason header).

A CPU-only engine reads its prompt slowly (measured: ~21.6 tokens/s for a 4B Q8_0 model, so 8 s covers only ~170 prompt tokens). The budget is **per request, not per prompt**. Size it as

```
sum over the request's questions of (prompt_tokens_q / prefill_tokens_per_second + decode_q) + queue wait + margin
```

because the letter-logit readout makes one sequential `/v1/chat/completions` call per question (`internal/gateway/letter.go`), each prompt carries the full state again, and in deterministic mode no prompt cache is used (`cache_prompt` is off), so nothing is reused between the questions. Deterministic mode also keeps one slot per engine instance, so other requests queued on the same instance add their own engine time to the queue wait. Worked example at 21.6 tokens/s: a request with three questions whose prompts are ~400 tokens each costs 3 x 400 / 21.6 = ~56 s of prefill before decode, queue wait and margin, so `LLMCTL_DECIDE_TIMEOUT=60` would only just cover it with an idle queue; one question with a ~1000-token prompt costs ~46 s. Raise the budget for such an engine accordingly. **Do not retry a `deadline_exceeded` 502**: each retry cancels the engine task and restarts the whole prefill, so it is guaranteed waste (the golden runner `scripts/golden/run_golden.py` therefore records it after one attempt; an `engine_error` 502 is still retried). The Go default is deliberately unchanged (8 s); raising it above the hosted SDK's 10 s is a choice made by the launcher for CPU engines (below) or by the operator.

CPU-adaptive default (launcher-side, G-156)

Measured on one CPU-only host (nezha.local, 8 threads, `decide-pro`, same pinned tree, same golden set): with the gateway default of 8 s, **98 of 132** golden requests were well-formed and **45 answers were HTTP 502**, every one carrying `x-llmctl-decide-reason: deadline_exceeded` and `x-llmctl-decide-deadline-ms: 8000`; with `LLMCTL_DECIDE_TIMEOUT=300` the same host returned **131 of 132** well-formed and **0** 502 (`specs/009-jev-decision-models/evidence/live-models/nezha-pinned-decide-pro-default8s-control-2026-10-09/` vs `nezha-pinned-decide-pro-2026-10-09/`). On that CPU the HTTP 200 latency is median 7.6 s (`decide-pro`, max 21.0 s) and 13.3 s (`decide-max`, 9B, p95 18.7 s, max 22.0 s): an 8 s budget cut about a third of the requests (45 of 132 golden requests, HTTP 502).

So the two ways the gateway is launched - the boot service (`lib/svc_hook.sh run-gateway`, the systemd unit / launchd agent) and `llmctl decide serve` (`lib/decide.sh`) - apply a **CPU-adaptive default** (`lib/decide_timeout.sh`):

- if `LLMCTL_DECIDE_TIMEOUT` is **not** set in the environment (including the unit's `gateway.conf` / the agent's environment), and
- at least one decision instance that is **running or enabled** has a CPU-only llama.cpp placement: the layer-offload flag (`--n-gpu-layers N`, `--n-gpu-layers=N`, `--gpu-layers N`, `--gpu-layers=N`, `-ngl N` or `-ngl=N`) has the value `0` (leading zeros allowed; when the flag is repeated the **last** occurrence wins, as in llama.cpp; partial offload, any value above 0, or no flag at all counts as not CPU). Read from the instance records under `$LLMCTL_SERVICES_DIR`. The CPU-only **onnx NLI encoder does not count**: measured on CPU it answers in a median 433 ms (p95 1585 ms, max 2837 ms, `evidence/live-models/nezha-pinned-decide-nli-2026-10-09/`), well inside 8 s, and counting it would raise the deadline on every GPU host that merely runs the encoder,

then the launcher exports `LLMCTL_DECIDE_TIMEOUT=120` (about 5 x the slowest measured answer, so a request that is merely slow completes while a wedged engine is still cut off in two minutes). The deadline is **one global value**: a single CPU llama instance raises it for **all** profiles, so a wedged GPU engine is then cut off at 120 s instead of 8 s. Per-profile timeouts (a deadline chosen

from the profile a request is routed to) are the future refinement; nothing implements them today. An **explicit `LLMCTL_DECIDE_TIMEOUT` always wins** (`8` stays `8`). A host whose decision engines are all on GPU keeps the 8 s default. Chat-only CPU instances do not count. The start-up banner shows the effective value and where it came from:

```
timeout: 8s per request (default; set LLMCTL_DECIDE_TIMEOUT to change)
timeout: 120s per request (cpu-adaptive: CPU-placed decide-pro; an explicit LLMCTL_DECI
timeout: 300s per request (env LLMCTL_DECIDE_TIMEOUT)
```

Stopping a gateway with slow requests in flight. On stop (SIGTERM/SIGINT) the gateway drains: readiness flips, new work is answered 503, and in-flight requests may finish for at most `LLMCTL_DECIDE_DRAIN_GRACE` (default **15 s**; the process waits that plus 2 s), after which remaining connections are closed. A CPU request can legitimately run for up to the 120 s adaptive deadline, so a request still running when the 15 s grace ends is cut off by the stop and the client sees a closed connection, not an answer. The grace is a separate tunable and is not adapted to the request deadline (gap G-161); raise `LLMCTL_DECIDE_DRAIN_GRACE` explicitly if stopping must wait for slow CPU requests (and give the unit/agent a stop timeout that is not shorter).

Limits: the value is decided when the gateway starts. A CPU instance started or enabled later does not change a running gateway - restart it (`llmctl decide serve --stop` then start, or restart the unit). The 120 s is a documented default derived from one host's measurements, not a guarantee: very long prompts still need the sizing above. `llmctl decide ask` has its own per-attempt client wait (30 s by default, `--timeout` / `LLMCTL_DECIDE_TIMEOUT` in the client's environment); a client waiting 30 s cannot use a gateway budget above that, so set it as well on a CPU deployment. Because the adaptive value exceeds the hosted SDK's 10 s timeout, an SDK client with the default 10 s still gives up first - that is the trade-off of serving CPU engines at all.

Connection admission (FR-022, "without affecting others")

Every connection starts **unauthenticated** and is promoted the first time a request on it carries the valid key.

- The global budget `LLMCTL_DECIDE_MAX_CONNS` (64) is split: at most `LLMCTL_DECIDE_MAX_UNAUTH_CONNS` (32) connections may be unauthenticated at once; the other slots are reserved for authenticated connections and hostile sources cannot take them.
- Per source (an IPv4 address, an IPv6 `/64`): `LLMCTL_DECIDE_MAX_CONNS_PER_SOURCE` (8) in total and `LLMCTL_DECIDE_MAX_UNAUTH_PER_SOURCE` (8) unauthenticated; an IPv6 `/48` shares `LLMCTL_DECIDE_MAX_UNAUTH_PER_PREFIX` (16) unauthenticated slots, so rotating addresses inside one allocation does not multiply a source's share.
- A connection that has not authenticated within `LLMCTL_DECIDE_PREAUTH_TIMEOUT` (3 s) of being accepted is reset - bare TCP, stalled handshakes, and an unauthenticated probe left idle alike. Authenticated keep-alive connections idle for `LLMCTL_DECIDE_IDLE_TIMEOUT` (60 s).
- A full unauthenticated pool does **not** refuse a newcomer: the oldest unauthenticated connection of the source that holds the most of them is reset and the newcomer takes its place, so a handful of hostile sources can only evict each other. A client with the valid key from another source is served (`TestValidKeyClientIsServedWhileHostileSourcesHoldTheSlots`).
- **Honest limits.** One source behind a NAT shares its per-source budget with everything else behind that address. A volumetric attacker that floods from more distinct addresses (IPv4) or `/64`s (IPv6) than the unauthenticated pool is large can still churn that pool and delay new connections; that is a network-layer problem and belongs to a firewall, a rate-limiting reverse proxy or the cloud provider's DDoS protection in front of the gateway. Raise the budgets for a busy shared front end; all of them are validated and fail the start when inconsistent.

Security notes

See `docs/tls-and-keys.md` for key strength, rotation and revocation, certificate-file checks, the placement guard and the loopback-engine caveat (a local user who binds an engine's port while the engine is down receives the internal key). `LLMCTL_DECIDE_TEMPERATURE` (readout scalar, default 1.0), `LLMCTL_DECIDE_MASS_THRESHOLD` and `LLMCTL_SEED` apply to the `letter-logit` driver. `LLMCTL_DECIDE_MODE=deterministic` (default: fixed seed, one slot per instance, byte-identical answers per instance) or `throughput`.

Calibration and the decision log

Applying a calibration profile (FR-080). `llmctl-decide calibrate` fits a recalibrator on operator-labelled answers and writes `$LLMCTL_STATE_DIR/decide/calibration/<profile>.json` (mode 0600), bound to the model file's sha256 and to the prompt template hash. The gateway loads that file for each decision profile **at start and on `SIGHUP`** (`kill -HUP $(cat $LLMCTL_STATE_DIR/decide/gateway.pid)` ; `llmctl decide serve` reloads the certificate pair on the same signal; there is no `cert reload` subcommand) - a profile written later is NOT picked up until then. It is applied only when all of this holds: the file is a regular, non-symlink file owned by the gateway's user and not writable by group/others; it is bound; its `model_sha256` equals the catalog's single `files[role=model].sha256` (the downloaded, checksum-verified file - the gateway does not re-hash a multi-GB model at start); its `template_hash` equals the live template hash; its parameters are well-formed. Otherwise the answers stay uncalibrated, `GET /v1/models` shows `"calibration":{"applied":false,"reason":"mismatch|unbound|invalid|insecure|model_unresolved"}` and the gateway writes ONE line to stderr naming the profile and the reason (again only if the reason changes).

What changes in an answer. Only `confidence`. The profile maps the winner's own probability (the largest listed `probabilities` value - the `p_pred` the labels were collected with) to a calibrated probability that the answer is correct; `confidence` then holds that value on the 0-1 probability scale (chance level is 1/n, **not** 0 as in the shaped convention - a `--min-confidence` threshold

chosen for the shaped value means something else on a calibrated answer), `confidence_raw` keeps the shaped value, and `calibration: {method, n, profile_id}` says what was applied. `probabilities`, `choice`, `score`, `flags` and `upper_bounds` are never touched, so the argmax and the raw readout stay auditable. `n < 200` supports no ECE claim (SC-003); `n` is published so a client can see it. A non-finite calibrator result leaves the answer uncalibrated. A `noul` answer has no confidence and is untouched.

Worst case. An answer with `flags:["option_missing"]` already carries the worst-case distribution (the absent options hold the mass that leaves the winner the smallest share). Calibration may **lower** such an answer's confidence but never raise it above that worst-case winner share: the answer must not claim more certainty than its readout supported. A fully listed answer is not capped, so an under-confident model can be calibrated upwards.

The template hash. `GET /v1/models` publishes each profile's `template_hash` (the gateway computes it; the catalog does not store one): a SHA-256 over the rendered prompt template, the readout spellings, `n_probs` and the effective `LLMCTL_DECIDE_TEMPERATURE` for letter-logit; over the forwarded hosted-shape body for native; over the hypothesis construction for the encoder. Changing the prompt, those parameters or the temperature invalidates calibration profiles fitted before - refit with `llmctl-decide calibrate` (it binds to the same computed value; no `--template-hash` needed).

The opt-in decision log (FR-080, 2-I05). Off unless asked for. `LLMCTL_DECIDE_DECISION_LOG=<path>` enables it **without** text; `LLMCTL_DECIDE_LOG_STATE=1` is the consent to keep the state and question text as well (and enables it at `$LLMCTL_LOG_DIR/decide-decisions.jsonl` if no path is given). One JSON line per decision request that was authenticated and parsed (including ones the backend then failed): UTC time, request id, profile, question counts by type, per question the type, the **index** of the winning option, the noul/score value, the served and raw confidence, `calibrated`, flags; latency, HTTP status, the model sha256, the template hash and the applied calibration profile. With consent each question also carries its name, instructions and the chosen option key, and the record carries the `state`; the start banner states whether text is logged. The Authorization header and keys have no field in a record. The file is created mode 0600 (parents 0700, symlinks

refused) through the FR-087 placement guard, so it is refused inside an unignored git work tree; it must not be the request log; it has no size cap or rotation of its own (an external rotation is followed). A state with consent is *the user's data*: treat the file accordingly and feed it, labelled, to `llmctl-decide calibrate`. Write failures never fail a request; they are counted in `llmctl_decide_audit_write_failures_total` and reported once on stderr.

Token-usage honesty

`usage.input_tokens` is an estimate, not a tokenizer-exact count; `usage.output_tokens` is exact for decoder profiles. Confidence is a shaping convention, not a calibrated probability of correctness (unless the answer carries `calibration`, see above). An answer with `flags: ["option_missing"]` had an option letter absent from the engine's readout: it is reported with an upper bound (never as an exact 0), and its probabilities are the **worst case** over what the absent options could hold. Derivation (the same text is in `internal/readout/readout.go`): the engine lists the top-n first tokens, so every unlisted token has probability at most `p_min` (the smallest listed one), and everything unlisted together holds `U = 1 - sum(listed)`. An option letter is the sum of up to `s = 3` spelling tokens (`B` , `B` , `▁B`), so an absent letter holds at most `min(s * p_min, U)` (a fully listed distribution, `U = 0`, bounds it by 0), and all absent letters TOGETHER hold at most `U`. The reported `probabilities`, `choice` and `confidence` use the allocation inside that polytope that leaves the winner the smallest share; the winner is always an option the engine **listed** (an absent option is never the answer). `--min-confidence` and MCP `min_confidence` gate on that listed winner's worst-case confidence; the bound is not applied a second time. Limits, stated: the masses of the *listed* letters are lower bounds (their own unlisted spellings are not added), and an engine exposing more than three token ids for one letter is not covered.

Budgets, errors and modes

- **Context budget.** Each profile's catalog `defaults.ctx` - or `LLMCTL_CTX_<PROFILE>` when set (the gateway applies the same override

and the same validation as the engine launcher; set it in the gateway's environment too) - is the context one request sees (the scheduler passes `--ctx-size ctx*parallel`; llama-server divides it over the slots - measured, `specs/009-jev-decision-models/evidence/review-2/ctx-measurements.json`). A request whose rendered prompt (state, question, options, template reserve) is *estimated* not to fit is a `422 validation_failed` before any engine is contacted; `GET /v1/models` advertises `limits.max_state_chars` and `limits.max_context_tokens`. The gateway also asks the engine for the per-slot context it really serves (`GET /props`, cached 10 s, dropped at once when the engine answers a request with an error, one fetch shared by concurrent requests) and uses the smaller value for both the refusal and the advertised `limits` (for `/v1/models`: the smallest over the healthy instances); without `/props` it falls back to the configured value. The `/props` read is the only engine contact before the refusal: the budget of **every** question of a request is checked before the first completion is requested. `LLMCTL_CTX_<PROFILE>` is read like Python's `int()` in the launcher (blanks, a sign and single underscores between digits, e.g. `4_096`, are accepted; non-ASCII digits, which Python also accepts, are refused by the gateway with a start error naming the variable); the gateway additionally caps it at 2^24. The estimate is per-script (digits and JSON cost more tokens per character than prose; private-use and rare extension-A ideographs are priced at their measured rate, the rare tail of the common CJK block is not covered by it); all measurements come from one tokenizer, so if the estimate is wrong the engine's own `exceed_context_size_error` is mapped to the same 422.

- **Encoder cost.** At most `LLMCTL_DECIDE_MAX_PAIRS` (default 64) premise/hypothesis pairs per request; an empty state is a 422.
- **Error classes.** `422` = this request cannot be served by this model (never retry); `500 backend_failed` = a deterministic server-side fault (bad readout setting, engine refusing the gateway's request shape; see the gateway log; never retry **for llmctl's own client** - the hosted SDKs retry every 5xx by default, so pass `RetryPolicy(http_statuses={408, 429, 502, 503, 529})` to them); `502` / `503` / `529` / `429` = transient (an engine that answers 404/405 is a stale registry entry **when the endpoint came**

from the registry: `502`; from a static endpoint it is a fixed misconfiguration: `500`, not retryable).

- **Mode.** Every decision answers `x-llmctl-decide-mode: deterministic|throughput`; deterministic mode requires `LLMCTL_DECIDE_SLOTS=1`, `readout.cache_prompt=false` in the catalog (a catalog that says otherwise is refused at start) and a `LLMCTL_SEED` of 0 or more (0 is a valid fixed seed; unset means 1). Byte-identity holds per instance: a busy primary overflows to the next instance, and every decision names the answering instance in `x-llmctl-decide-instance` (client `evidence.instance`).
- **Question text is data.** Instructions, option keys, labels and the state are neutralised before they reach the prompt (control and bidi characters removed, line breaks in keys/labels collapsed, marker-like lines escaped). The check for marker-like lines is applied on a *folded* copy of each line, used for MATCHING only (the line itself keeps its bytes and gets a `|` prefix): compatibility decomposition (full-width and circled forms count); every character of the categories Cf (zero-width, bidi, soft hyphen, BOM, variation selectors), Cc, Mn/Me (combining and enclosing marks) and Zs/Zl/Zp (all blanks, anywhere in the line) dropped, plus the invisible non-Cf fillers (Hangul fillers U+115F/U+1160/U+3164/U+FFA0, the Braille blank U+2800, U+180E); tag characters mapped to the ASCII they spell; any dash (Unicode Pd), U+2212, bullets and markdown emphasis/quote/heading marks ignored as a list prefix; and a documented table of look-alikes (Greek, Cyrillic, Turkish dotless i, Latin small capitals, Armenian, Cherokee, Lisu) mapped to Latin. Persian/Indic joiners, emoji ZWJ sequences, Korean and the Mongolian vowel separator are kept in content (they are never removed from a line that is not a marker). **No such table is complete**: it is best effort, not a guarantee, and a look-alike outside it is ordinary content, still inside the delimited block and below the template's own header.

Finding engines

By default (`LLMCTL_DECIDE_RESOLVER=auto`) the gateway routes to the engines in the service registry when it lists any, else to the static `LLMCTL_DECIDE_ENDPOINT_<PROFILE>` - and, only with `LLMCTL_VALIDATE_STRICT=0`, to the `LLMCTL_PORT_<PROFILE>` / catalog port

of a profile without a healthy registry row (review 2026-10-15 finding 7: under the strict validation gate an unregistered engine is not a backend). A profile counts as declared only when `LLMCTL_DECIDE_ENDPOINT_<PROFILE>` names at least ONE non-empty URL: a value of `,` or `" , ,"` is no declaration (review 3 of 2026-10-15, finding 8 - it used to bring the catalog-port fallback back). `LLMCTL_DECIDE_RESOLVER=static` is an explicit operator choice and keeps the full static table below; since review 3 the gateway prints a startup WARNING that static mode routes to catalog ports / declared URLs WITHOUT the validation gate (an engine there is validated only if you made sure of it: `llmctl require-validated <profile>`). It publishes itself in the registry (`kind=gateway`) while serving.

The running gateway binary (review 3, finding 11 - documented, nothing was rebuilt or restarted). Measured read-only on the review host on 2026-10-15: `build/llmctl-decide` is dated 2026-10-14 20:39, before `a745432` (finding 7 of the second review, committed 2026-10-15 12:50), and the live `llmctl-decide-gateway.service` process runs an executable that has since been replaced (`readlink /proc/<MainPID>/exe` -> `.../build/llmctl-decide (deleted)`). Neither the finding-7 strict fallback nor the finding-8 fix above is in effect in that process until the operator rebuilds (`llmctl build decide`) and restarts the gateway; this fix round deliberately did not touch live services. Check a host with `readlink /proc/$(systemctl --user show -p MainPID --value llmctl-decide-gateway.service)/exe` and compare the binary's build with `git log -1`. See `docs/registry-discovery.md`.

The static endpoint of a profile is, in order: `LLMCTL_DECIDE_ENDPOINT_<PROFILE>` (comma separated loopback URLs) > `http://127.0.0.1:$LLMCTL_PORT_<PROFILE>` > `http://127.0.0.1:<catalog port>`. `LLMCTL_PORT_<PROFILE>` is the same host-local port rebind the shell side honours (`<PROFILE>` upper-cased, `-` -> `_`, e.g. `LLMCTL_PORT_DECIDE_NLI=18096`), so a hand-started engine on an overridden port is found without also setting the endpoint variable. A value that is not an integer in 1-65535 (including `auto`) is ignored and the catalog port is used. In `auto` and `registry` mode a healthy registry entry for the profile is preferred over these static endpoints.

`auto` **decides per profile, on every request.** A gateway started in `auto` mode *before* any decision engine is registered (engines take up to `LLMCTL_REGISTER_WAIT`, default 600 s, to load their model and register) serves the static endpoints, and a profile switches to the registry the moment it has a healthy registry entry (and back to its static endpoint when that entry goes away or turns unhealthy) - no restart (C2-13, B3-09). A profile served only statically keeps working next to a registered sibling. The boot unit forces `LLMCTL_DECIDE_RESOLVER=registry` (`svc_hook.sh run-gateway`).

One-question smoke (`llmctl decide smoke`)

`llmctl decide smoke --url URL --protocol letter-logit|nli-onnx|systemone-native [--key-file F] [--options N] [--expect-choice KEY] [--json]` asks ONE engine a fixed, deterministic choice question through the production driver and exits **0 only for a valid typed answer** (finite probabilities summing to 1, and `--expect-choice` if given); 1 backend failure, 2 usage, 6 engine unreachable. `lib/download.sh` runs it after downloading a decision GGUF.

Typed-question schema and MCP server (for coding agents)

- `llmctl decide schema [--format json-schema|openai-tool|mcp]` prints the typed-question schema as a tool definition (default `json-schema`). It needs no network and no key.
- `build/llmctl-decide mcp` is a minimal local MCP server over stdio with one tool, `decide` . It reads the same endpoint, CA and key as `ask` (`--endpoint` , `--cacert` , `--retries`). A configuration problem (no key, bad CA) does **not** stop the server: every call then returns a tool error that says why (`isError: true`), so an agent can never read a failure as an answer. The tool-level `min_confidence` withholds a low-confidence answer as an error. The tool description states that the model is not a chat model and that confidence is a shaping convention.

- The shell front end does not forward `mcp` (nor `smoke`, `vantage`) yet: run the binary directly (`LLMCTL_DECIDE_BIN` locates it). Agent wiring: agents.